CHAMPIONTRACKPRO V2: MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

EXECUTIVE SUMMARY

You are acting as the Lead Full-Stack Engineer and Cloud Security Architect for "ChampionTrackPro", a B2B SaaS targeting NCAA D2/D3 Basketball programs.

We are moving from a V1 MVP to a V2 Enterprise-grade product.

Strategic Pillars:

1. **Zero Medical Liability (HIPAA Avoidance):** We track "Wellness" and "Friction", never "Pain" or clinical "Injuries".
2. **Coach Full Access:** Since data is non-medical, Head Coaches require 100% read access to their roster's data to make immediate decisions.
3. **Frictionless UX:** Player input must take < 15 seconds.
4. **Actionable Insights:** We replace raw data spaghetti charts with aggregated Readiness Scores and Exponential Moving Average (EMA) trendlines.

PHASE 1: RED TEAM AUDIT (CURRENT STATE ANALYSIS)

Before writing new code, execute a ruthless audit of the current V1 workspace focusing on Firebase Auth, Firestore rules, and client-side logic.

Provide an Executive Audit Report detailing:

- 🟢 **Strengths:** Correct implementations in the current codebase.

- 🔴 **CRITICAL VULNERABILITIES:** Focus on exposed data (`allow read: if true`), IDOR vulnerabilities, and PII mixed with performance data.
- 🟡 **Architecture Flaws:** Bottlenecks, client-side data filtering instead of server-side querying, and orphaned document issues on account deletion.
- 🛠️ **Remediation Plan:** Prioritized checklist of exact files to modify.

PHASE 2: DATABASE ARCHITECTURE & SECURITY

Design a NoSQL Firestore schema that strictly separates PII from performance data, ensuring scalability and security.

1. Data Schema Requirements:

- `users`: Stores PII (`userId`, `name`, `email`, `role`, `teamId`, `position`, `birthYear`).
- `daily_logs`: Stores the daily metrics. Documents must include `userId`, `teamId`, `createdAt` (timestamp), and the metric scores. **CRITICAL:** Do not duplicate PII in this collection.
- `ai_training_dataset`: An isolated Data Lake for anonymized historical logs.

2. Firebase Security Rules (firestore.rules):

- **Authentication:** Reject all unauthenticated requests.
- **Player Role (`role == 'player'`):** Can only READ and WRITE their own documents in `users` and `daily_logs` where `resource.data.userId == request.auth.uid`.
- **Coach Role (`role == 'coach'`):** Can READ all documents in `users` and `daily_logs` STRICTLY IF the document's `teamId` matches the coach's `teamId`. Cannot write/edit player logs.
- **AI Dataset:** The `ai_training_dataset` must be locked down (`allow read, write: if false;`), accessible only via Cloud Functions (Admin SDK).

PHASE 3: THE PLAYER UI (DAILY QUESTIONNAIRE)

Build the `<DailyWellnessQuestionnaire>` component.

1. Strict UI/UX Guidelines:

- **Theme:** Premium Dark Mode (Deep Navy/Black backgrounds). Primary action color is Cyan (`#00E5FF`).
- **The `<SemanticSlider>` Component:** * Absolutely NO math symbols (`+` or) and no static numbers visible on the track.
  - Rely solely on semantic text anchors (`leftAnchorText`, `rightAnchorText`) in muted gray at extremities.
  - Numeric value (1-10) appears ONLY inside a floating tooltip while the user actively drags the thumb. Default state must require user interaction.

2. Data Model (JSON Payload):

Section A: The Daily Baseline (5 Sliders, mapped 1-10)

- `cardioLoad`: "How did your lungs handle the pace today?" (Left: Never out of breath | Right: Completely Gassed)
- `neuroLoad`: "How bouncy and explosive did your legs feel?" (Left: Bouncy / Light | Right: Heavy / Glued to the floor)
- `sleepQuality`: "How restorative was your sleep last night?" (Left: Terrible / Restless | Right: Deep / Woke up fresh)
- `tacticalIQ`: "How sharp was your decision-making and playbook execution?" (Left: Lost / A step behind | Right: Locked-in / Reading the floor)
- `teamChemistry`: "How connected did you feel with the team's energy today?" (Left: Disconnected / Frustrated | Right: Total chemistry / Great energy)

Section B: The Friction Matrix (Conditional)

- `hasFriction` (Boolean): "Are you currently experiencing any specific friction limiting your performance?" ([NO] / [YES])
- *(If YES, reveal)*:
  - `frictionType` (Select): "Physical Soreness", "Academic/Life Stress", "Court Confusion", "Mental/Emotional"
  - `frictionFrequency` (Slider): "How often is this distracting you?" (Rarely ➔ Constantly)

    - `frictionImpact` (Slider): "How much is this limiting your ability to play at 100%?" (Barely noticeable ➔ Severely limiting)
    - `frictionDistraction` (Slider): "Honestly, how much is this issue messing with your head right now?" (Not worried ➔ Highly distracted)

PHASE 4: THE COACH DASHBOARD (PERFORMANCE ANALYTICS)

Transition the dashboard from a "Data Viewer" to a "Decision Support Tool". Retain the Dark Mode, Control Bar (Filters), and Global Navigation.

1. The "Morning Brief" (Triage System):

Create a parent view sorting the roster by risk. Implement a traffic light system (🔴 Danger, 🟡 Monitor, 🟢 Optimal). At-risk players must render at the top via summary cards.

2. The Champion Readiness Score:

Write a utility to aggregate the daily scores into a single Readiness Score (0-100). Display this via a circular progress gauge.

3. The Radar Chart (Root-Cause Analysis):

Implement a Recharts `<RadarChart>` grouping indicators into Physical, Mental, and Technical categories to instantly visualize skewed performance states.

4. EMA Contextualization (Trend Deviation Math):

Write a TypeScript utility for calculating the Exponential Moving Average (EMA) and Deviation.

- Smoothing factor for a window N (use $N=28$): $\alpha = \frac{2}{N + 1}$
- EMA on day t: $EMA_t = (V_t \times \alpha) + (EMA_{t-1} \times (1 - \alpha))$
- Deviation percentage: $Deviation = \frac{V_t - EMA_t}{EMA_t} \times 100$
- *Missing Data:* If V_t is null, carry forward EMA_{t-1} to maintain an unbroken trendline.

5. Charting Components:

- **Deviation Bar Chart (`<ComposedChart>`):** Y-Axis [1, 10]. EMA 28 as a subtle solid line. Raw values as `<Bar>`.
    - Colors: >+15% (Red), +8% to +15% (Orange), -8% to +8% (Green), <-8% (Blue).
- **Workload Area Chart (`<ComposedChart>`):** Acute Load (7-day EMA) as a thick cyan line. Chronic Load (28-day EMA) as a dashed blue line. Implement a semi-transparent red "Danger Zone" `<Area>` fill when Acute > Chronic. Custom tooltips required.

PHASE 5: AI DATA LAKE RETENTION (CLOUD FUNCTION)

Write a Firebase Cloud Function (`anonymizePlayerDataForAI` in TypeScript).

- **Trigger:** When a user is deleted from the `users` collection.
- **Logic:** Query all matching `daily_logs`. Strip identifiers (`userId`, `teamId`). Inject metadata (`position`, `sport`, `ageAtLog`). Write anonymized logs to `ai_training_dataset` via batched writes (`db.batch()`). Hard delete original logs to comply with CCPA/GDPR.

PHASE 6: FINAL HANDOVER & IMPLEMENTATION REPORT

Upon successful completion, testing, and approval of all prior phases, you must generate a comprehensive technical documentation file named `ChampionTrackPro_Implementation_Report.md` in the root directory.

This document will serve as the single source of truth for the V2 architecture.

The report must include the following sections:

1. **Executive Summary:** A brief recap of the V2 features successfully implemented.
2. **File Directory Map:** A structured list of all new and modified files (Components, Hooks, Utility functions, Firebase configs).
3. **Database Schema (Final State):** The exact JSON/TypeScript interfaces deployed to Firestore.

4. **Security Audit Confirmation:** A summary of the deployed `firestore.rules` and how they enforce the Team Isolation and Player-only access.
5. **Mathematical Implementations:** The final TypeScript formulas used for the Readiness Score and the EMA/Deviation calculations.
6. **Technical Debt & Recommendations:** Any edge cases encountered, hardcoded values that need dynamic environments, or recommendations for Phase V3 (e.g., performance optimizations, query indexing).

Execution Command: Do not generate this report until I explicitly say: *"All phases are complete, please generate the handover report."*

CLAUDE CODE EXECUTION PROTOCOL

Read this document thoroughly. Acknowledge understanding of the strategic constraints. Execute tasks sequentially. Do not proceed to the next phase until the current phase is fully coded, reviewed, and approved by the user.
